AGH

WEAIiIB ISI

Projekt 1: easyAI – Nim i Nimby

Przedmiot: Inteligencja obliczeniowa

Autorzy:	Mikołaj Dudkiewicz
	Eliza Petrycka
Grupa:	[6]

19 marca 2026

Spis treści

1 Opis zadania i gry **1**

2 Implementacja **1**

3 Metodologia eksperymentow **1**

4 Wyniki – zadania na 4 punkty **2**

5 Wyniki – zadania na 6 punktow **2**

6 Wyniki – zadania na 8 punktow **3**

7 Napotkane problemy **4**

8 Reprodukowanie wynikow **4**

1 Opis zadania i gry

W ramach laboratorium zrealizowano wszystkie punkty z pliku `EasyAI.pdf`. Wybrana gra probabilistyczna to **Nimby**. Zakres obejmuje:

- implementacje wariantu probabilistycznego gry,
- turnieje AI vs AI z kontrola gracza rozpoczynajacego,
- porownanie algorytmow Negamax (z i bez odciecia alfa-beta),
- pomiar czasu podejmowania decyzji,
- implementacje wariantu expecti-minimax z odcieciem alfa-beta.

Gra bazowa to klasyczny Nim (wariant misere): gracz zabierajacy ostatni element przegrywa. Stan poczatkowy w eksperymentach: trzy stosy $(3, 4, 5)$. Wariant Nimby wprowadza szum wykonania: dla ruchu "zabierz k" (gdy $k > 1$), z prawdopodobienstwem 0.1 wykonywane jest "zabierz $k - 1$".

2 Implementacja

Kod znajduje sie w pliku `lab_01/nimby_experiments.py`. Zaimplementowano:

- klase `NimbyGame` (deterministyczny Nim i probabilistyczny Nimby przez parametr `error_probability`),
- `NegamaxNoAB` – Negamax bez odciecia alfa-beta,
- `ExpectiNegamaxAB` – wariant expecti-minimax z odcieciem alfa-beta,
- `TimedAIPlayer` – zliczanie czasu decyzji i liczby ruchow,
- runner eksperymentow zapisujacy wyniki do `lab_01/results.json`.

Wazny aspekt implementacyjny. W srodowisku probabilistycznym nie mozna tracic spojnosci miedzy `make_move` a `unmake_move`. Dlatego rzeczywista liczba zabranych elementow jest zapisywana na stosie historii i cofana 1:1. Dla expecti-minimax zamiast losowania uzyto jawnych wezlow szansy (normalny ruch vs "slip") i wartosci oczekiwanej:

$$E = (1 - p)\, V_{\text{normal}} + p\, V_{\text{slip}}, \quad p = 0.1.$$

Wykresy do raportu sa generowane automatycznie przez skrypt `lab_01/generate_plots.py` na podstawie `lab_01/results.json`.

3 Metodologia eksperymentow

Wszystkie testy uruchomiono skryptem:

```
uv run python lab_01/nimby_experiments.py
```

Wspolne ustawienia:

- naprzemienne rozpoczynanie partii (eliminacja biasu pierwszego ruchu),
- limit 200 polruchow na partie (w praktyce brak remisow),
- te same stany poczatkowe i kontrola ziaren losowosci po numerze partii.

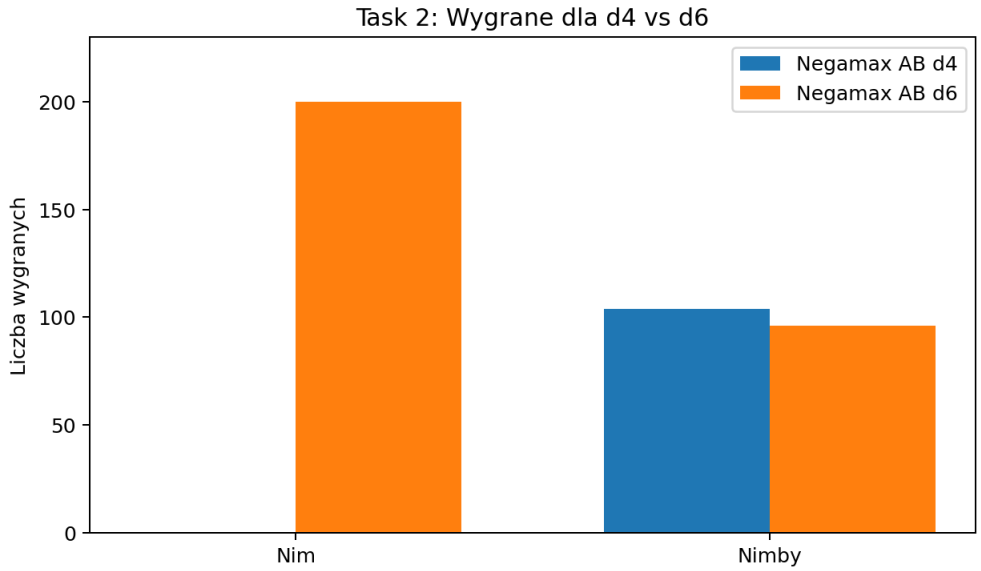
4 Wyniki – zadania na 4 punkty

Porownano dwa ustawienia glebokosci Negamax: `depth=4` i `depth=6`, osobno dla Nim (deterministyczny) i Nimby (probabilistyczny).

Tabela 1: NegamaxAB depth 4 vs depth 6 (200 partii)

Wariant	Wygrane d4	Wygrane d6	Remisy	Sredni czas ruchu [ms] d4 : d6
Nim (det.)	0	200	0	0.7146 : 5.4021
Nimby (prob.)	104	96	0	0.8147 : 6.0359

Komentarz. W wariancie deterministycznym glebsze przeszukiwanie dominuje calkowicie. W wariancie probabilistycznym roznica glebokosci przestaje byc monotonicznie korzystna: przypadkowosc wykonania ruchu zaburza planowanie klasycznego Negamax.



Rysunek 1: Task 2: liczba wygranych dla NegamaxAB d4 i d6.

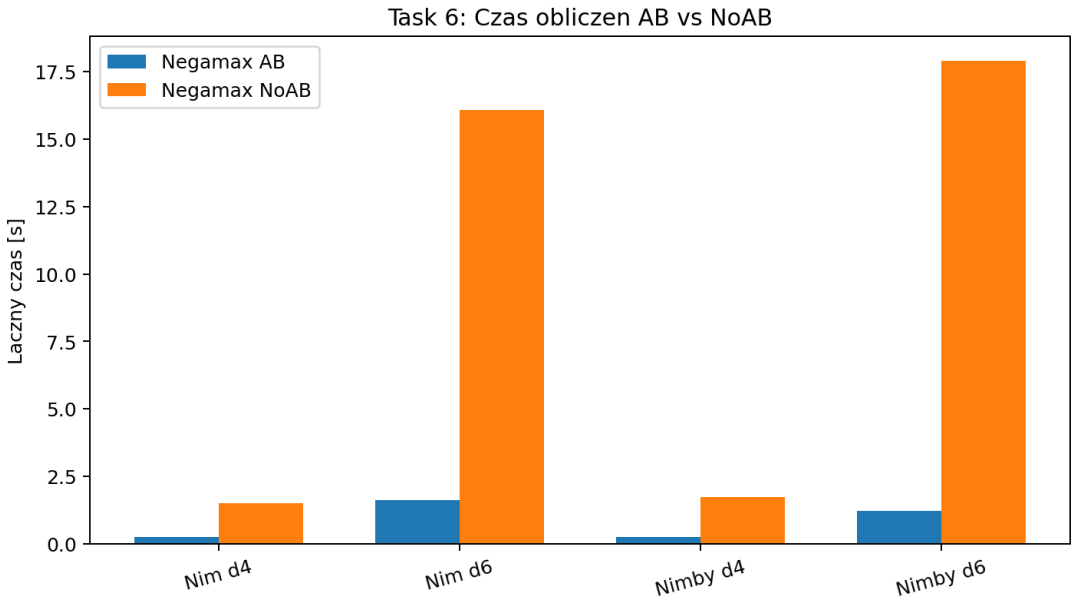
5 Wyniki – zadania na 6 punktow

Wykonano porownanie Negamax z i bez odciecia alfa-beta, dla dwoch glebokosci (4, 6) i dla obu wariantow gry.

Tabela 2: Negamax z/bez alfa-beta (100 partii na kazde starcie)

Wariant	Starcie	Wygrane A	Wygrane B	Czas A [s]	Czas B [s]
Nim	AB d4 vs NoAB d4	50	50	0.3694	2.1376
Nim	AB d6 vs NoAB d6	50	50	2.3406	23.1431
Nimby	AB d4 vs NoAB d4	59	41	0.3349	2.3374
Nimby	AB d6 vs NoAB d6	51	49	1.8052	25.6892

Wnioski. Decyzje strategiczne AB i NoAB sa porownywalne (w Nim idealnie identyczne), ale wydajnosc czasowa AB jest wielokrotnie lepsza. Dla `depth=6` bez odciecia koszt obliczen jest rzedu 10× wiekszy.



Rysunek 2: Task 6: porownanie lacznego czasu obliczen AB vs NoAB.

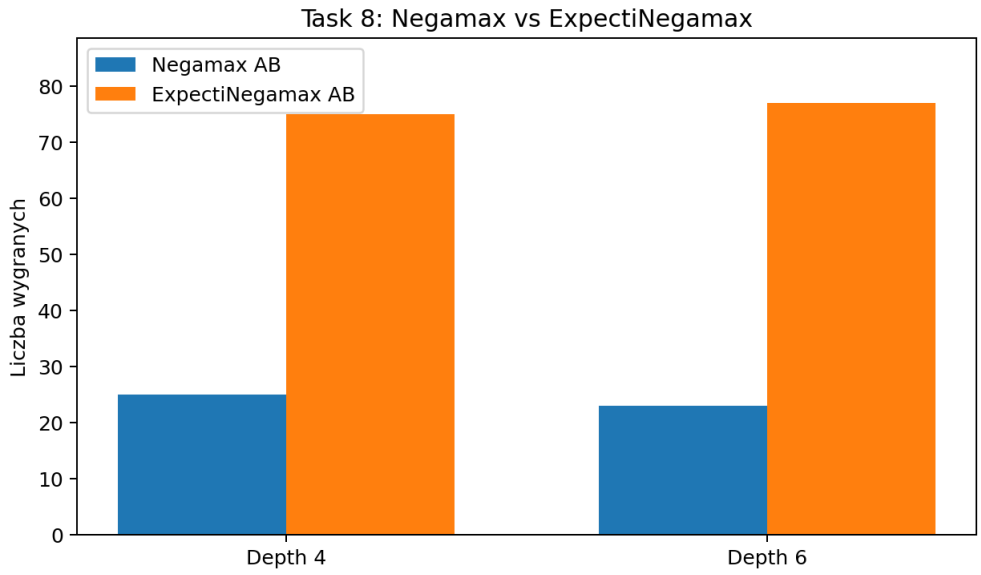
6 Wyniki – zadania na 8 punktow

Zaimplementowano expecti-minimax z odcieciem alfa-beta i porownano z klasycznym NegamaxAB w Nimby.

Tabela 3: NegamaxAB vs ExpectiNegamaxAB (Nimby, 100 partii)

Starcie	Wygrane Negamax	Wygrane Expecti	Czas Negamax [s]	Czas Expecti [s]
Depth 4	25	75	0.3544	0.9316
Depth 6	23	77	2.1210	11.0509

Wnioski koncowe. Expecti-minimax wyraznie poprawia skutecznosc w grze probabilistycznej, bo modeluje ryzyko bledu wykonania ruchu. Cena za te poprawe to wyzszy czas obliczen (okolo 2.6× dla depth 4 i ponad 5× dla depth 6).



Rysunek 3: Task 8: NegamaxAB kontra ExpectiNegamaxAB (Nimby).

7 Napotkane problemy

Glownym problemem byla zgodnosc logiki probabilistycznej z mechanizmem przeszukiwania drzewa. Negamax zaklada spojnosc wykonania/cofania ruchu; przy naiwnej losowosci w `make_move` mozna latwo uszkodzic stan gry. Problem rozwiazano przez:

- jawna rejestracje rzeczywiscie wykonanego ruchu,
- deterministyczne cofanie ruchow po historii,
- osobne modelowanie wezlow szansy w expecti-minimax.

8 Reprodukowanie wynikow

- generowanie wykresow: `uv run python lab_01/generate_plots.py`
- uruchomienie: `uv run python lab_01/nimby_experiments.py`
- dane wyjsciowe: `lab_01/results.json`
- raport: `lab_01/report.tex`